

NAME: K. HARSHIKA

REG.NO:192473020

COURSE CODE: CSA0609

ASSESSMENT 3

Debugging Bubble Sort Code (Inner Loop Uses Fixed Upper Bound)

The following Bubble Sort implementation always runs the inner loop up to the last index, even though the largest elements have already settled at the end after each pass. Carefully analyze why this does not affect correctness in every case but still introduces inefficiency and can increase the risk of unnecessary comparisons. Debug the code to use the proper shrinking inner loop boundary. Optimize the implementation further using a swap flag to stop early when the array becomes sorted. Analyze the corrected algorithm in best, average, and worst cases and rewrite the final version with comments.

```
def bubble_sort(arr):  
    n = len(arr)  
    for i in range(n):  
        for j in range(0, n-1): # ISSUE  
            if arr[j] > arr[j+1]:  
                arr[j], arr[j+1] = arr[j+1], arr[j]  
    return arr
```

ANSWER:

First Error:

```
for j in range(0, n-1):
```

What is the error?

The inner loop always runs until n-1 in every pass.

In Bubble Sort, after each outer loop pass (i), the largest element is already placed in its correct position at the end of the array. Therefore, there is no need to compare that sorted part again.

Why is it an error?

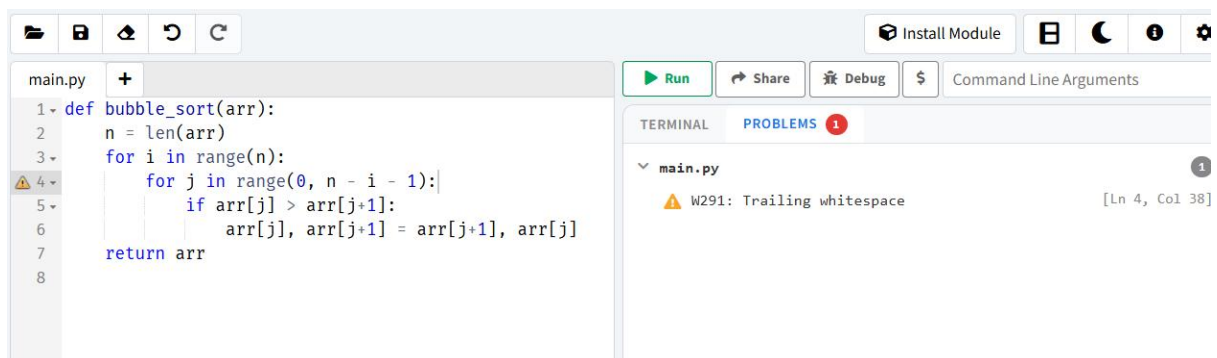
It causes unnecessary comparisons, making the algorithm less efficient.

Correct Fix:

```
for j in range(0, n - i - 1):
```

After Fixing the Error

```
def bubble_sort(arr):  
    n = len(arr)  
    for i in range(n):  
        for j in range(0, n - i - 1):  
            if arr[j] > arr[j+1]:  
                arr[j], arr[j+1] = arr[j+1], arr[j]  
    return arr
```



Second Error:

```
for i in range(n):
```

What is the error?

The algorithm always completes all n passes, even if the array becomes sorted before all passes are finished.

Why is it an error?

If the array is already sorted or becomes sorted after a few passes, the remaining passes do not make any changes. The algorithm still keeps running, which causes unnecessary iterations and reduces efficiency.

Correct Fix

```
for i in range(n):  
    swapped = False
```

After Fixing the Error

```
def bubble_sort(arr):  
    n = len(arr)  
    for i in range(n):  
        for j in range(0, n - i - 1): # Fixed  
            if arr[j] > arr[j+1]:  
                arr[j], arr[j+1] = arr[j+1], arr[j]  
    return arr
```

Final Code

```
def bubble_sort(arr):  
    n = len(arr)  
    for i in range(n):  
        swapped = False  
        for j in range(0, n - i - 1):  
            if arr[j] > arr[j+1]:  
                arr[j], arr[j+1] = arr[j+1], arr[j]
```

```

        swapped = True

    if not swapped:
        break

    return arr

arr = [5, 3, 8, 4, 2]
print(bubble_sort(arr))

```

Output

The screenshot shows a code editor with a file named 'main.py'. The code implements a bubble sort function. The function 'bubble_sort(arr)' takes an array and sorts it in ascending order. It uses a 'swapped' flag to optimize the process by breaking out of the loop if no swaps occur in a pass. The array [5, 3, 8, 4, 2] is passed to the function, and the sorted array [2, 3, 4, 5, 8] is printed. The terminal output on the right confirms the sorted array and shows the process exited with a return code of 0.

```

main.py  +
1 def bubble_sort(arr):
2     n = len(arr)
3
4     for i in range(n):
5         swapped = False
6
7         for j in range(0, n - i - 1):
8             if arr[j] > arr[j+1]:
9                 arr[j], arr[j+1] = arr[j+1], arr[j]
10                swapped = True
11
12        if not swapped:
13            break
14
15    return arr
16
17
18 arr = [5, 3, 8, 4, 2]
19
20 print(bubble_sort(arr))

```

Run Share Debug Command Line Arguments

TERMINAL PROBLEMS 1

[2, 3, 4, 5, 8]

** Process exited - Return Code: 0 **

Time Complexity

BestCase:

The best case occurs when the input array is already sorted. During the first pass, no swaps are made, so the algorithm stops immediately using the swapped flag. Therefore, the time complexity is $O(n)$ because it only checks each element once.

AverageCase:

The average case occurs when the elements are in random order. The algorithm performs multiple passes and comparisons before the array becomes sorted. Therefore, the time complexity is $O(n^2)$.

WorstCase:

The worst case occurs when the array is in reverse order. Every comparison results in a swap, and all passes are required to sort the array. Therefore, the time complexity is $O(n^2)$.

Algorithm

1. Start.
2. Input the array to be sorted.
3. Find the number of elements in the array and store it in n .
4. Repeat the sorting process for each pass from 0 to $n-1$.
5. Set a variable swapped to False at the beginning of each pass.
6. Compare each pair of adjacent elements only in the unsorted portion of the array (0 to $n-i-2$).
7. If the left element is greater than the right element, swap them and set swapped = True.
8. Continue comparing until the end of the unsorted portion.
9. If no swaps occurred during the current pass (swapped is False), stop the algorithm because the array is already sorted.
10. Otherwise, continue with the next pass.
11. Repeat until all passes are completed or the array becomes sorted.
12. Display the sorted array.
13. Stop.

Pseudocode

BEGIN

Read the array

$n \leftarrow$ length of the array

FOR each pass i from 0 to $n-1$

 swapped $\leftarrow$ False

 FOR each index j from 0 to $n-i-2$

 IF $\text{arr}[j] > \text{arr}[j+1]$ THEN

 Swap $\text{arr}[j]$ and $\text{arr}[j+1]$

 swapped $\leftarrow$ True

 END IF

 END FOR

IF swapped = False THEN

EXIT the loop

END IF

END FOR

Display the sorted array

END